

Activation Oracles

Training and Evaluating LLMs as General-Purpose Activation Explainers

Karvonen, A., Chua, J., Dumas, C., Fraser-Taliente, K., Kantamneni, S., Minder, J., Ong, E., Sen Sharma, A., Wen, D., Evans, O., & Marks, S. (2026). *Activation Oracles: Training and Evaluating LLMs as General-Purpose Activation Explainers*. *arXiv:2512.15674 [cs.CL]*. <https://arxiv.org/abs/2512.15674>

Huang, V., Choi, D., Johnson, D. D., Schwettmann, S., & Steinhardt, J. (2025). *Predictive Concept Decoders: Training Scalable End-to-End Interpretability Assistants*. *arXiv:2512.15712 [cs.AI]*. <https://arxiv.org/abs/2512.15712>

Pan, A., Chen, L., & Steinhardt, J. (2024). *LatentQA: Teaching LLMs to Decode Activations Into Natural Language*. *arXiv:2412.08686 [cs.LG]*. <https://arxiv.org/abs/2412.08686>

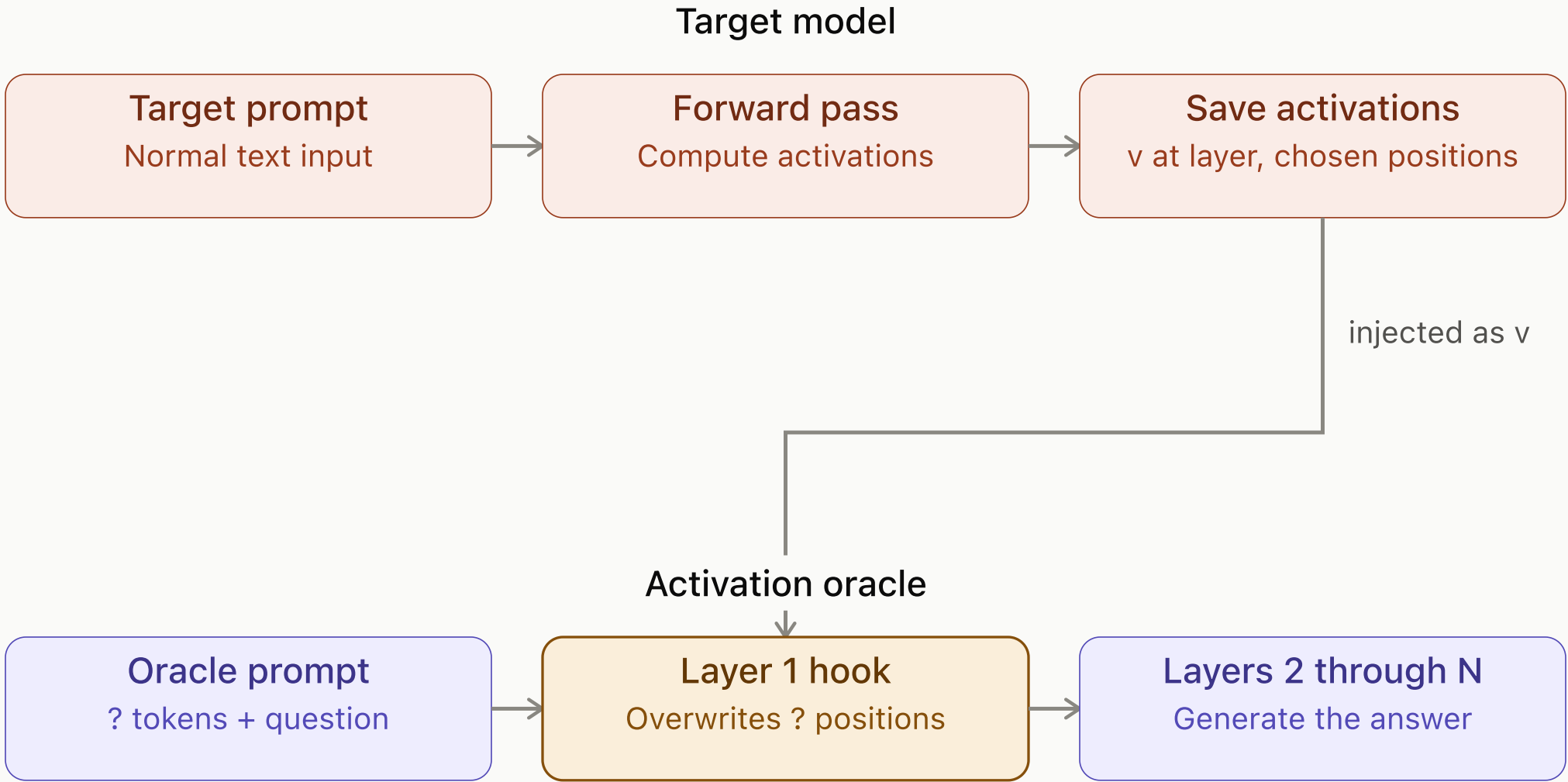
How Activation Oracles are trained

- **Target vs. suspect model:** "target model" is the general role — whoever's activations get captured; a "suspect model" is a target that's been fine-tuned to hide something, used specifically for the auditing evals
- **Key asymmetry:** the oracle is trained *only* on activations from the clean base model — **never** on activations from any fine-tuned/suspect model — yet still generalizes to auditing them
- **Setup:** a LoRA adapter ($r=64$) on the *same base architecture* as the target it will audit — trained for Llama-3.3-70B-Instruct, Gemma-2-9B-IT, Qwen3-8B, and Claude Haiku 3.5 (full fine-tune, not LoRA)
- **Training data — 3 task types mixed together:**
 - System Prompt QA (SPQA) — infer system-prompt properties from conversation activations
 - Classification — 7 binary NLP tasks (sentiment, language ID, entailment, etc.)
 - Self-supervised context prediction — predict surrounding tokens from an activation sequence, no input/target overlap
- **Scale:** ~1M examples, activations from 3 depths (25/50/75%) of the *base* model; evaluated at 50% depth; 10 H100-hrs (Qwen3-8B) / 90 H200-hrs (Llama-3.3-70B)

What Activation Oracles are used for

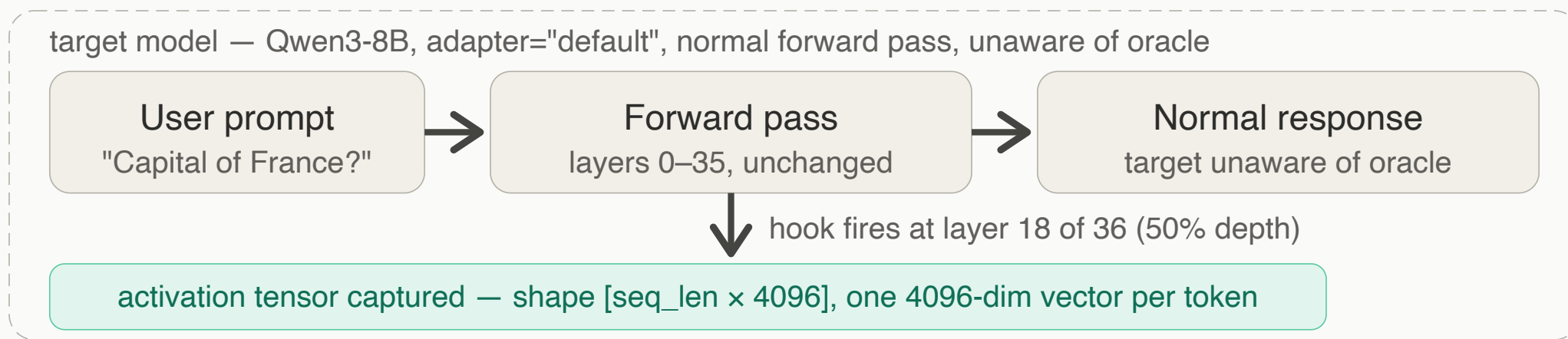
- Recovering information fine-tuned into a model — biographical facts, malign propensities — that never appears in the input text, even though the oracle was never trained on activations from that fine-tuned model
- **Secret-keeping** (Section 4, "auditing games"): extracting hidden words, gender assumptions, or hidden instructions (Taboo, User Gender, SSC) — flagship demo exposes both a secret goal and a secret word from a model fine-tuned to play "Taboo," purely from activations, not outputs
- **Emergent misalignment audits:** describing what's driving a narrowly misaligned model, given base/fine-tuned activation differences

Injection mechanism — how target activations enter the oracle



Only the ? token positions are overwritten — every other token passes through unchanged

Steps 1 & 2 — Collect activations and build the oracle prompt



Step 1 Collect activations

```
collect_activations_multiple_layers(model, submodules, inputs_BL)
```

- registers forward hook on `model.model.layers[18]`

- hook stores `outputs[0]` into a dict, then raises `EarlyStopException` to stop early

Step 2 Build oracle prompt + find ? positions

```
get_introspection_prefix(layer=18, num_positions=5)
```

- returns `"Layer: 18\n ? ? ? ? ? \n"` prepended to the user question

```
find_pattern_in_tokens(token_ids, " ?", n, tokenizer)
```

- scans tokenised prompt for all `" ?"` IDs, verifies count = n and consecutive, returns positions

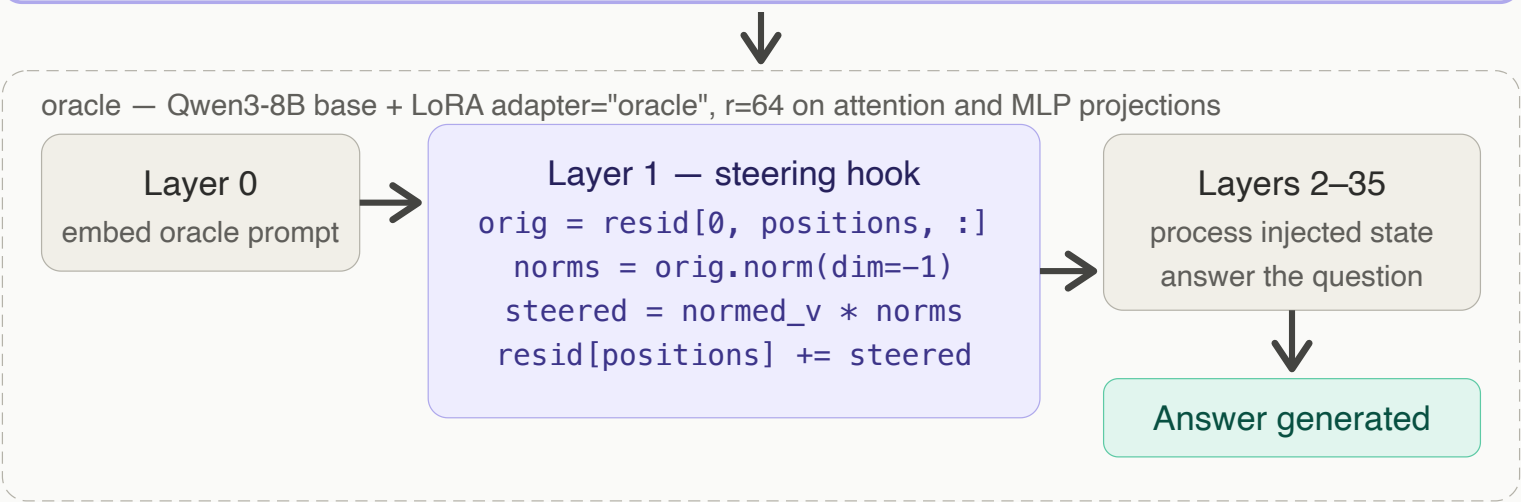
Steps 3–5 — Package, inject and generate

Step 3 Package into OracleInput

```
create_oracle_input(prompt, layer, n, tokenizer, acts_BD)
```

— returns an `OracleInput` dataclass with four fields:

- `input_ids` — tokenised oracle prompt
- `steering_vectors` — activations cloned to CPU, detached from graph



Layer 1 formula Norm-matching injection — Eq. 1

$$h'_i = h_i + \|h_i\| \cdot (v_i / \|v_i\|)$$

v_i — target activation, reduced to a unit vector (direction only, magnitude discarded)
 h_i — oracle's own activation at that position; supplies the magnitude to rescale to
rescaled v_i is added to h_i , never replaces it — replacement caused a norm blow-up
20× at layer 0 · 100,000× at layer 1 — addition avoids this entirely

LoRA (low-rank adaptation)

- Base weight $\mathbf{W}$ ($d \times k$) stays frozen — never updated during training
- Two small trainable matrices instead: $\mathbf{A}$ ($r \times k$) and $\mathbf{B}$ ($d \times r$), with $r \ll d$ — the oracle uses $r=64$
- Update: $\Delta\mathbf{W} = \mathbf{B} \cdot \mathbf{A}$, merged at inference as $\mathbf{W}' = \mathbf{W} + \Delta\mathbf{W}$
- Applied to `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`
- The oracle itself is loaded as a LoRA adapter on the *same base architecture* as the target model

Worked example ($d = 4096$, $r = 64$)

- Frozen weight $\mathbf{W}$: $4096 \times 4096 \approx \mathbf{16.8M}$ parameters
- Trainable $\mathbf{A}$: 64×4096 , trainable $\mathbf{B}$: $4096 \times 64 \rightarrow$ combined $\approx \mathbf{524k}$ parameters
- That's roughly **32× fewer** trainable parameters than fine-tuning $\mathbf{W}$ directly
- Same idea applies per module — `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj` each get their own small $\mathbf{A/B}$ pair at $r = 64$

Norm-matching injection formula (Eq. 1)

$$\mathbf{h}'_i = \mathbf{h}_i + \|\mathbf{h}_i\| \cdot \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|}$$

- $\mathbf{v}_i$ — target activation, reduced to a unit vector (direction only); $\mathbf{h}_i$ — oracle's own activation, supplies the magnitude
- **Norm** $\|v\|$ = vector length, e.g. $v=[3,4] \rightarrow \|v\|=5$; **unit vector** $v/\|v\| = [0.6, 0.8]$ (length 1, same direction)
- **Rescale**: $\|h\| \cdot$ unit vector — e.g. if $\|h_i\|=12$, result is $12 \cdot [0.6, 0.8] = [7.2, 9.6]$, norm exactly 12, pointing in $\mathbf{v}_i$'s direction
- Result is **added** to $\mathbf{h}_i$, never replaces it — replacement caused **20x** (layer 0) / **100,000x** (layer 1) norm blow-up
- Why norm-match: raw activations from different sources (residual stream, SAE features, diffs) have wildly different magnitudes — rescaling to $\|\mathbf{h}_i\|$ keeps every injection at a scale the oracle already expects, so only *direction* carries new information